

Lesson 0008 — Evaluation and fine-tuning

~85–110 MIN

Evaluation

ANCHORED TO: [DOCS/EVALUATION.MD](#) · [EVALUATION/M5_LIVE/REPORT.MD](#) · [DOCS/M3D-REVIVED-PLAN.MD](#)

Measure before changing. This lesson turns the current M5 evidence into an evaluation loop and separates a planned fine-tune from a verified deployment result.

HOW TO USE THIS LESSON

Read the explanation, inspect the visual, make a prediction, and then compare it with the repository evidence. The goal is a model of this project that you can explain—not a list of framework slogans.

PREREQUISITES AND SCOPE

This lesson builds on training/validation from Lesson 0003 and the failure-localization method from Lesson 0004. It teaches how to design evidence; it does not claim that a planned fine-tune has already succeeded.

LEARNING OBJECTIVES

By the end, you should be able to define the unit and denominator of a metric, match predictions to labelled objects without double counting, separate threshold selection from final testing, recognize temporal leakage, design a paired baseline-versus-candidate test, and explain why fine-tuning is an experiment rather than an automatic improvement.

Measure the question you actually asked

1 · ACCURACY

Needs labelled truth:

class + box + IoU match

Then compute precision, recall, AP, or a defined frame metric.

2 · RUNTIME COUNTS

m5_analyze_bag.py counts:

keeps + class IDs + rates

Useful coverage diagnostics.

Not ground-truth accuracy.

3 · FUTURE PLAN

Capture table totals 80:

mixed 3-cube, single-cube, occlusion, edge, held-out, and background-only frames.

Other plan prose totals 88.
Reconcile before capture.

EVIDENCE LOOP

Define gate → collect labelled cases → match predictions to truth → change one cause → rerun → promote or roll back.

Corrected overview — keep counts are not accuracy, and the future capture split must be reconciled before use. The former portrait sheet is superseded; the source SVG remains available.

Retrieval warm-up

Answer from memory before revealing the cues. Difficulty here is useful: it strengthens retrieval and shows what deserves review.

1. For z-depth, should a raised cube top normally be nearer or farther than the floor behind it?

► Reveal recall cues

2. What can the geometry filter check, and what can it never create?

► Reveal recall cues

The question this lesson answers

How do we decide whether a detector is good enough, and how do we improve it without confusing a plausible change with a verified fix? The answer is an evidence loop: define a gate, capture a controlled case, analyze the result, change one cause, and rerun the same gate with rollback available.

STATUS BOUNDARY

The M3d-revived plan is a recommendation, not completed evidence. It proposes a positive-domain fine-tune because M5 is PARTIAL; no new model should be described as successful until its regression and live gates pass.

GROUND TRUTH COMES BEFORE METRICS

Ground truth is the human-reviewed answer: which cubes exist, their classes, and their boxes. A prediction becomes a true positive only after class agreement and sufficient box overlap with an unmatched ground-truth object. An unmatched prediction is a false positive; an unmatched ground-truth cube is a false negative. Without this matching step, message counts and “frames with a detection” are operational observations—not detection accuracy.

Matching is **one-to-one**. If two red predictions overlap one labelled red cube, the higher-ranked matching prediction may be one true positive, but the duplicate cannot claim the same cube again; it becomes a false positive. Likewise, one wide prediction cannot satisfy two labelled cubes. This rule is what makes duplicate predictions affect precision instead of silently inflating success.

1. Define “success” before collecting data

The project definition names three high-level targets: at least 80% color-classification accuracy on a structured 50-frame test, at least 5 fps inference on the Jetson, and an operating distance of 20–80 cm. The M5 acceptance report adds stricter implementation gates such as an empty-scene zero-keep result, three cube classes in the cubes bag, and a 25 Hz frame-loop target. These are related but not interchangeable measurements.

Every target needs a **unit** and a **denominator**. “80%” could mean correctly classified labelled cubes, frames where every cube is correct, or class occurrences across a bag; those produce different numbers. Write the protocol as a testable sentence—for example, “at least 40 of 50 preselected frames contain a correctly localized and classified target at IoU ≥ 0.50 and confidence ≥ 0.50 .” If that is not the intended definition, choose another one before collecting evidence.

≥80%

structured 50-frame color
accuracy target

≥5 fps

project-level inference target

0 / 3

current M5 cubes keeps at conf
0.50 / classes

2. Use a metric that matches the question

$\text{precision} = \text{TP} / (\text{TP} + \text{FP})$ and $\text{recall} = \text{TP} / (\text{TP} + \text{FN})$. Raising the confidence threshold commonly removes both weak false positives and weak true positives, so precision may rise while recall falls. IoU supplies the box-overlap matching rule. Average Precision summarizes the precision–recall curve for one class; mAP averages AP across classes and, depending on the named variant, across one or several IoU thresholds. Always state the exact variant and dataset.

Object-level and frame-level metrics answer different questions. One frame containing three cubes can contribute three object decisions, but only one all-or-nothing frame decision. Per-class results also matter: an overall average can hide that green works while blue never appears. For empty scenes, report false detections per frame or the fraction of empty frames with any false detection; ordinary precision is not informative when there are no true objects.

Question	Useful measurement	Do not substitute
Is detection/classification correct?	ground-truth matching by class and IoU, then precision/recall/AP or a precisely defined frame-level accuracy	class occurrence or keep count
Does each class ever appear when expected?	per-class frame-hit and keep count as a coverage diagnostic	accuracy
Does the empty scene stay quiet?	false keeps on empty/distractor frames	validation mAP alone
Is the node responsive?	publish/frame-loop rate and p50/p95 latency	GPU-only benchmark qps
Did the model retain training knowledge?	held-out Roboflow validation mAP	training loss alone

ANALYZER BOUNDARY

`m5_analyze_bag.py` counts published class IDs; it does not load ground-truth boxes or perform IoU matching. Its keeps and class occurrences are useful runtime diagnostics, but false positives can increase them. Do not report those counts as accuracy.

THRESHOLDS BELONG TO VALIDATION

A confidence threshold and an NMS IoU threshold are hyperparameters. Sweep or choose them on training/validation evidence, then freeze them before the final test. Choosing the threshold that looks best on the final test leaks test information into the design and makes the reported result optimistic.

3. Build a test matrix, not a single demo

Vary one important factor at a time when possible—class, distance, angle, lighting, background, empty scene—while preserving a named baseline. Otherwise a **confound** makes attribution impossible: if the model, threshold, room, and camera angle all change together, a better result does not tell you which change helped.

A useful matrix names the object classes, scene type, distance, viewpoint, lighting, and expected outcome before looking at the predictions. Empty scenes measure false positives. Cubes-in-frame scenes measure positives. Repeated frames at the same setup reveal whether a result is stable rather than a lucky single frame.

Choose the sampling unit carefully. Adjacent video frames are highly correlated: 100 consecutive frames may be closer to one observation repeated 100 times than to 100 independent scenes. Split and report by capture session, physical arrangement, or time block where possible. When comparing a baseline and candidate model, run both on the **same frozen inputs**; otherwise scene changes can masquerade as model improvements.

SCENE	20 cm	40 cm	60 cm	80 cm
empty	0 keep			
1R+1G+1B		3 classes		
distractor			0 keep	

same protocol · comparable evidence

The proposed M6 distance bands are useful only when capture and labeling preserve the context of each frame.

4. Capture evidence with provenance

The repository's capture scripts save images or ROS bags alongside metadata and SHA-256 checks. `capture_frames.py` is the RGB saver; `capture_rgb_depth_sync.py` handles synchronized RGB/depth pairs. Integrity metadata tells us what was analyzed; it does not make the labels correct by itself.

A reproducible run should identify the model/ONNX/engine hashes, code revision, parameter file and effective thresholds, dataset or bag hash, label version, camera setup, and random seed where randomness exists. A filename such as `best_v2.pt` is convenient but not sufficient identity. Labels also need a written policy for partial cubes, occlusion, image-edge truncation, and ambiguous colors so two annotators do not silently create different ground truth.

5. Replay and analyze instead of eyeballing one frame

`m5_analyze_bag.py` extracts per-class kept counts, publish rate, RGB rate, and other summary evidence from a bag. `m5_parse_latency.py` parses latency summaries. This separation turns a long run into numbers that can be compared across model versions. [\[recorded M5 results\]](#)

Interactive: choose the next action after a miss

Repeat the same capture and confirm the miss is stable before changing the model.

For timing, discard or separately report warm-up frames, then give a distribution such as p50 and p95 latency—not only one average. Throughput and latency are related but different: a pipeline may process many frames per second in batches while one individual frame still waits too long. Measure the end-to-end path relevant to the robot as well as isolated inference time.

6. Read the current M5 result as a matrix

Bag / setting	Messages	Total kept	What it tells us
Empty, conf 0.50	439	0	observed empty-scene false-keep result; physical geometry reason unvalidated
Cubes sticker-on, conf 0.50	414	0	positive gate fails

Bag / setting	Messages	Total kept	What it tells us
Cubes sticker-off, conf 0.50	460	0	sticker hypothesis did not explain failure
Cubes sticker-off, conf 0.25	439	2 green	threshold-only rescue is insufficient

The current live rate is also measurable: roughly 15.2–15.9 Hz for the node frame loop, with about 26.6 ms YOLO time. Keep this separate from the project-level 5 fps target and from correctness.

7. Acceptance gates are conjunctive

The M3d-revived plan proposes that a candidate must preserve empty-scene behavior, keep Roboflow validation mAP@0.5 at or above 0.80, and meet the cubes-in-frame bar of at least three keeps covering all three classes at confidence 0.50. Its geometry regression must be treated as behavior-only until the reversed depth sign is corrected; after correction, define a physically valid geometry gate. A candidate that fixes blue but regresses empty scenes is not a pass. “Better on one screenshot” is not a gate.

The “three keeps covering all three classes” condition is a useful **smoke gate**, not proof of 80% accuracy or robust detection. It can pass with one lucky frame, badly placed boxes, or extra false positives unless the protocol also specifies labelled objects, one-to-one matches, allowed false positives, frame count, and conditions. Keep the smoke gate, but do not promote it into a stronger claim than it measures.

8. Fine-tuning is a targeted intervention

A **hard negative** is a non-cube scene or object that the current detector mistakes for a cube; training on it can reduce that specific false-positive pattern. A **positive-domain example** shows a real target under deployment conditions and can address misses. Class imbalance means one class or condition contributes much more learning evidence than another. A useful dataset therefore follows the measured error pattern instead of merely adding more easy images.

The recommended plan targets 80 new JetRover-room RGB frames, but not all frames contain all three cubes. Its capture table includes 48 three-cube distance-band frames, 4 lighting frames, 9 single-cube frames, 4 partial-occlusion frames after trimming, 4 background-only frames, 3 edge-angle frames, and 8 held-out M5c2-layout replicates. The prose immediately above that table still says every frame has exactly three cubes. A later dataset table separately claims 76

positive training frames, 4 background training frames, and 8 held-out replicates—88 new frames rather than 80. These are planning defects; reconcile the exact split before collection or training.

Only after that specification is reconciled does the plan continue from `models/best.pt` for 30 epochs at `lr=0.0005`, image size 640, batch 16, and seed 42. The purpose is to test whether representative room-domain data improves positive detections—not to assume that it will, replace YOLOv5, add a fourth class, or lower the production threshold indefinitely.

Thirty epochs and the proposed learning rate are starting hypotheses, not guarantees against overfitting. Monitor training and validation curves, preserve the best validation checkpoint, and stop or revise the experiment if room performance rises while the original validation domain collapses. Augmentation can create controlled variations of existing images, but it cannot invent every missing camera, material, lighting, or background condition.

Data selection should follow the localized error. Add representative positives for missed room cubes; add hard negatives only for repeatable false-positive patterns; balance classes and difficult conditions; and avoid flooding training with near-duplicate easy frames. A candidate can improve one domain while suffering **catastrophic forgetting** on an older domain, which is why both original and room-domain held-out gates remain necessary.

Before retraining: reconcile the class mapping. The current model, normalized dataset, and node use `0=blue_cube`, `1=green_cube`, `2=red_cube`, while `docs/project-definition.md` still lists `red/green/blue`. A fine-tune with an unexamined mapping could make otherwise plausible metrics meaningless.

PLANNED, NOT YET EXECUTED:

```
python scripts/finetune_jetrover_positives.py \\  
  --data data/jetrover_v2/data.yaml --base models/best.pt \\  
  --out models/best_v2.pt --epochs 30 --batch 16 \\  
  --imgsz 640 --lr 0.0005 --device 0 --patience 20
```

The planned script `scripts/finetune_jetrover_positives.py` is not present in the current checkout. This block documents the plan; it is not a command to run as if implementation already existed.

9. Protect the validation signal

Near-duplicate video frames can leak the same scene into training and validation and make metrics look stronger than true generalization. Split by capture session or scene when possible. Keep a final test or acceptance set untouched by repeated threshold and hyperparameter choices; once you repeatedly tune against it, it has effectively become another validation set.

The proposed merged dataset keeps the original Roboflow validation images untouched, adds held-out JetRover replicates as a room-domain validation set, and preserves a test split. The plan's target composition is 170 train images, 17 validation images, and 4 test images. Those numbers are proposed future dataset counts, not current artifacts.

Also verify the capture-tool boundary before a physical session: the plan's example verifier syntax does not match the current `verify_camera_samples.py` CLI, which expects a positional directory plus `--prefix`. Reconcile the exact capture and verification commands before collecting data.

Small held-out sets have high uncertainty. Four test images can reveal obvious breakage, but one changed result moves an image-level percentage by 25 points. Report raw counts beside percentages and avoid treating tiny differences as proof of a general improvement. Collect more independent scenes when the decision matters.

10. Re-export and regression-test the entire chain

After a fine-tune, the plan requires a new ONNX export and checker, TensorRT engine construction on the Jetson, ONNX smoke inference, geometry regression, and M5 live bags. The geometry regression must include the corrected sign and labelled raised/floor cases; merely reproducing current KEEP/REJECT counts would preserve the defect. Changing `best.pt` without re-exporting and re-running the downstream checks would leave the deployed engine stale.

11. Rollback is part of responsible improvement

The current `models/best.pt` is preserved unchanged. A candidate such as `best_v2.pt` remains separate until the acceptance gate passes. If it fails, point the configuration back to the preserved artifact and record the outcome. This makes a model experiment reversible and keeps the production baseline identifiable.

12. Check your understanding

Why use more than one evaluation context?

1. ☐ Different contexts make every evaluation metric directly interchangeable.
2. ☐ Different contexts expose different failure modes and measurement needs.
3. ☐ Different contexts remove the need to record model versions.

What is the status of the M3d–revived fine–tune?

1. ☐ It is the model currently deployed in production.
2. ☐ It is the geometry filter currently evaluating candidates.
3. ☐ It is a proposed intervention that remains unexecuted.

When is a candidate ready for promotion?

1. ☐ After every defined regression and acceptance gate has passed.
2. ☐ After training loss improves on one selected training batch.
3. ☐ After its filename and creation timestamp are both newer.

What makes a baseline–versus–candidate comparison attributable?

1. ☐ Capture a convenient new scene for each model.
2. ☐ Evaluate both models on the same frozen labelled inputs and protocol.
3. ☐ Compare only their training losses.

13. Tiny read-only exercise

First calculate a miniature labelled result: suppose the detector publishes 8 boxes, 6 match different ground-truth cubes, and 2 match nothing; the scene contains 10 labelled cubes. Then $TP=6$, $FP=2$, $FN=4$, precision is $6/8=0.75$, and recall is $6/10=0.60$. This hand calculation is the conceptual prerequisite for interpreting any automated evaluator.

Open `evaluation/evaluate.py` before attempting to use it. Its current `main()` contains only `pass`, so it is a documented placeholder rather than a working evaluator. Use the recorded M5 report and the `capture/analyzer` scripts as the current evidence sources.

14. Explain it back

Explain this without reading: “Why is fine-tuning a hypothesis-driven experiment rather than a magic accuracy button?”

Include the current M5 evidence, one proposed data change, one held-out gate, and the rollback point.

What this lesson intentionally postpones

This lesson intentionally postpones implementing the future evaluation/fine-tuning scripts, choosing new hyperparameters beyond the recorded plan, and collecting physical robot data. It teaches how to make those decisions responsibly.

It also postpones statistical confidence intervals, calibration methods, annotation tooling, and the mathematics of AP integration. Those are not silently assumed here: they become useful after a larger labelled evaluation set and a working evaluator exist. Lesson 0009 rehearses how to explain the chosen experiment, its evidence, and its unresolved risks.

Primary references and next lesson

- COCO detection evaluation — the primary benchmark definition for IoU-based AP/AR evaluation.
- Ultralytics performance-metrics guide — practical interpretation of precision, recall, IoU, and mAP.
- Curated learning resources — includes the verified local Szeliski copy; Section 4.1.2 grounds the training/validation and overfitting discussion.
- Project README and the linked canonical documentation.
- Curriculum plan — scope, teaching contract, and project truth.
- Lesson 0009 — Project Walkthrough and Interview Practice: explain the complete system and own its limits.

CONTINUE WITH THE TEACHING AGENT

Ask the agent to clarify anything uncertain, challenge your explanation with a counterexample, or assess your explain-back. A learning record should be created only after you demonstrate the idea in your own words.

[← Previous lesson](#)[Course home](#)[Quick reference](#)[Mission](#)[Next lesson →](#)

